

UNIVERSIDAD PÚBLICA DE EL ALTO

INGENIERÍA DE SISTEMAS



PROYECTO FINAL

PREDICCIÓN CLIMÁTICA MEDIANTE RED NEURONAL

Estudiante:

Univ: Jhonatan Ramos Collquehuanca

Materia: INTELEGENCIA ARTIFICIAL II

Gestión: 2026

Paralelo: 9C -NOCHE

EL ALTO-BOLIVIA

2026



ÍNDICE

INTRODUCCIÓN.....	3
1. IDENTIFICACIÓN	3
1.1 ESTUDIO BIBLIOGRÁFICO	3
1.2 PLANTEAMIENTO DEL PROBLEMA	4
1.3 OBJETIVOS.....	4
1.3.1 Objetivo general	4
1.3.2 Objetivos específicos	5
2. DESARROLLO DEL MODELO	5
2.1 ELECCIÓN DEL MODELO	5
2.2 SELECCIÓN DE LA HERRAMIENTA	6
3. CONSTRUCCIÓN DEL MODELO	7
3.1 IDENTIFICACIÓN DE BARIBALES DE ENTRADA Y SALIDA.....	7
3.2 CONSTRUCCIÓN DE LOS CONJUNTOS DE ENTRENAMIENTO	8
3.3 ARQUITECTURA DEL MODELO	9
3.4 ENTRENAMIENTO (TRAI) Y PRUEBA	10
3.5 ANÁLISIS DE LOS RESULTADOS Y TOPOLOGÍA FINAL DE LA RED NEURONAL	16
4. CONSULTAS	24
5. TRANSFERENCIA TECNOLOGÍA.....	27
6. CONCLUSIONES	29
BIBLIOGRAFÍA	29



INTRODUCCIÓN

El clima siempre ha sido un factor determinante en la vida cotidiana y en el desarrollo de las ciudades. Contar con información confiable sobre las condiciones atmosféricas permite planificar actividades, prevenir riesgos y aprovechar mejor los recursos disponibles. En el caso de La Paz y El Alto, donde las variaciones de temperatura, humedad y radiación solar son notorias a lo largo del año, disponer de predicciones precisas resulta especialmente útil.

En este proyecto se propone el uso de **redes neuronales artificiales** como una herramienta moderna para la predicción climática. A diferencia de los métodos tradicionales, las redes neuronales tienen la capacidad de aprender patrones complejos a partir de grandes volúmenes de datos, lo que las convierte en una alternativa poderosa para anticipar variables como la temperatura, la humedad, el viento o incluso estados del clima como días soleados o lluviosos.

El trabajo se centra en la construcción de un modelo que, a partir de datos históricos obtenidos de un archivo climático (EPW), pueda estimar valores futuros y ofrecer resultados confiables. Además, se busca que este modelo no solo quede en un entorno académico, sino que pueda integrarse en un sistema web interactivo, accesible para cualquier usuario que desee consultar predicciones de manera sencilla y visual.

De esta manera, el proyecto combina la teoría de las redes neuronales con una aplicación práctica, aportando una solución tecnológica que puede servir como base para futuros desarrollos en el área de la meteorología y la inteligencia artificial aplicada.

1. IDENTIFICACIÓN

1.1 ESTUDIO BIBLIOGRÁFICO

El uso de redes neuronales para la predicción climática ha sido ampliamente explorado en distintos contextos. Diversas investigaciones muestran cómo los modelos de aprendizaje automático pueden anticipar variables como la temperatura, la radiación solar o la probabilidad de lluvia, logrando resultados alentadores en cuanto a precisión y utilidad. Plataformas académicas como Kaggle y repositorios especializados en datos abiertos han servido como base para estos estudios, al proveer conjuntos de datos históricos que permiten entrenar y validar modelos de inteligencia artificial.



En nuestro caso, el proyecto se apoya en un archivo climático en formato **EPW (EnergyPlus Weather File)** correspondiente a la región de La Paz–El Alto. Este dataset fue obtenido de la plataforma **Climate.OneBuilding.org**, un repositorio internacional que recopila y distribuye datos meteorológicos estandarizados para diferentes ciudades del mundo. La dirección utilizada para la descarga del data set fue:

https://climate.onebuilding.org/WMO_Region_3_South_America/BOL_Bolivia/LP_La_Paz/BOL_LP_La.Paz-El.Alto.Intl.AP.852010_TMYx.zip

La inclusión de esta fuente garantiza la calidad y confiabilidad de la información, lo que resulta fundamental para el entrenamiento de la red neuronal.

La revisión de estos antecedentes confirma que la predicción climática mediante inteligencia artificial es una línea de investigación válida y con resultados prometedores. Al mismo tiempo, nos motiva a adaptar estas técnicas a la realidad local de La Paz y El Alto, donde las condiciones geográficas y atmosféricas presentan particularidades que requieren modelos ajustados a su contexto.

1.2 PLANTEAMIENTO DEL PROBLEMA

En ciudades como La Paz y El Alto, el clima presenta variaciones bruscas que influyen directamente en la vida cotidiana, en la planificación de actividades y en sectores como la agricultura, la construcción o el transporte. Sin embargo, los métodos tradicionales de predicción climática suelen ser limitados, poco accesibles o no están adaptados a las particularidades locales.

La falta de herramientas prácticas y confiables genera incertidumbre en la población y en quienes dependen de condiciones atmosféricas estables para tomar decisiones. Surge entonces la necesidad de contar con un modelo que, a partir de datos históricos y variables meteorológicas, pueda ofrecer estimaciones más precisas y fáciles de interpretar.

El problema central radica en transformar información técnica y extensa en predicciones útiles, comprensibles y disponibles para cualquier usuario. La solución propuesta es el uso de redes neuronales artificiales, capaces de aprender patrones complejos y generar resultados que superen las limitaciones de los métodos convencionales.

1.3 OBJETIVOS

1.3.1 Objetivo general



El propósito central de este proyecto es diseñar y poner en práctica un modelo de red neuronal que permita realizar predicciones climáticas confiables para la región de La Paz–El Alto. La idea es aprovechar datos históricos y transformarlos en información útil, que pueda ser consultada de manera sencilla y aplicada en distintos ámbitos de la vida cotidiana y profesional.

1.3.2 Objetivos específicos

- **Predecir la temperatura ambiente** utilizando como referencia variables como la radiación solar, la humedad relativa, la velocidad del viento y la precipitación. Esto permitirá contar con un indicador básico pero fundamental para la planificación de actividades.
- **Extender las predicciones a otras variables climáticas**, como la humedad relativa, la radiación solar, la velocidad del viento, la precipitación y el punto de rocío. De esta manera, el modelo no se limita a una sola salida, sino que ofrece un panorama más completo de las condiciones atmosféricas.
- **Clasificar estados del clima** en categorías simples y comprensibles para cualquier usuario, como días soleados, nublados, lluviosos o con heladas. Esta clasificación facilita la interpretación de los resultados y los acerca a un lenguaje cotidiano.
- **Integrar el modelo en un sistema web interactivo**, que permita realizar consultas en tiempo real y visualizar las predicciones de manera clara y atractiva. El objetivo es que la herramienta no quede solo en el ámbito académico, sino que pueda ser utilizada de forma práctica por cualquier persona interesada en conocer el clima de su ciudad.

2. DESARROLLO DEL MODELO

2.1 ELECCIÓN DEL MODELO

Para el desarrollo de este proyecto se optó por utilizar un **Perceptrón Multicapa (MLP)**, que es una de las arquitecturas más comunes y efectivas dentro de las redes neuronales artificiales. La elección no fue casual: este tipo de modelo se caracteriza por su capacidad de aprender relaciones complejas entre las variables de entrada y producir resultados precisos en problemas de predicción.

El MLP está compuesto por varias capas de neuronas interconectadas, lo que le permite procesar la información de manera progresiva. A diferencia de un perceptrón



simple, que solo puede resolver problemas lineales, el perceptrón multicapa es capaz de identificar patrones no lineales, lo cual resulta fundamental en el caso del clima, donde las variables se relacionan de forma dinámica y muchas veces impredecible.

La decisión de trabajar con esta arquitectura responde a la necesidad de contar con un modelo flexible, robusto y adaptable a diferentes tipos de datos. Además, el MLP es ampliamente utilizado en investigaciones similares, lo que garantiza que su aplicación en este proyecto se encuentra respaldada por experiencias previas y resultados comprobados.

En resumen, el Perceptrón Multicapa fue elegido porque ofrece un equilibrio entre simplicidad y potencia, permitiendo construir un modelo que pueda aprender de los datos históricos y generar predicciones confiables para la región de La Paz–El Alto.

2.2 SELECCIÓN DE LA HERRAMIENTA

Para llevar adelante este proyecto se decidió trabajar con el lenguaje de programación **Python**, debido a que ofrece una amplia variedad de librerías especializadas en el manejo de datos y en la construcción de modelos de inteligencia artificial. Su versatilidad y facilidad de uso lo convierten en una de las herramientas más utilizadas en el ámbito académico y profesional.

Entre las librerías empleadas destacan **Pandas** para la manipulación de datos, **Scikit-learn** para el preprocesamiento y normalización, y **TensorFlow/Keras** para la creación y entrenamiento de la red neuronal. Estas herramientas permiten estructurar el flujo de trabajo de manera ordenada, desde la preparación de los datos hasta la obtención de resultados finales.

Además, se utilizó **Matplotlib** para la generación de gráficas que ilustran el desempeño del modelo, lo que facilita la interpretación de los resultados. El entorno de desarrollo elegido fue **Visual Studio Code (VSCode)**, por su practicidad y compatibilidad con múltiples extensiones que agilizan la programación y depuración del código.

La combinación de estas herramientas asegura un proceso eficiente y confiable, permitiendo que el modelo pueda ser entrenado, evaluado y posteriormente integrado en un sistema web interactivo. De esta manera, la selección tecnológica responde tanto a la necesidad de contar con recursos robustos como a la intención de que el proyecto sea accesible y replicable en futuros trabajos.



3. CONSTRUCCIÓN DEL MODELO

3.1 IDENTIFICACIÓN DE BARIBALES DE ENTRADA Y SALIDA

Para que la red neuronal pueda realizar predicciones confiables, es necesario definir con claridad cuáles serán las **variables de entrada** y cuál será la **variable de salida**. En este proyecto, las entradas corresponden a factores climáticos que influyen directamente en la temperatura y en otros estados del clima.

Las **variables de entrada** consideradas son:

- **Mes y hora:** permiten capturar la influencia de la estacionalidad y del ciclo diario en el comportamiento del clima.
- **Temperatura de rocío:** refleja la cantidad de humedad presente en el aire.
- **Humedad relativa:** porcentaje de vapor de agua en la atmósfera.
- **Radiación solar:** energía recibida en la superficie, clave para estimar variaciones térmicas.
- **Velocidad y dirección del viento:** factores que influyen en la dispersión de masas de aire y en la sensación térmica.
- **Precipitación:** cantidad de agua caída en un periodo determinado.

La **variable de salida principal** es la **temperatura ambiente**, ya que constituye el indicador más representativo y útil para la población. En versiones extendidas del modelo, también se busca predecir otras variables como la humedad relativa, la radiación solar o la velocidad del viento, con el fin de ofrecer un panorama más completo de las condiciones atmosféricas.



C9		fx	
	A	B	
	year,month,day,hour,dry_bulb_temperature_c,dew_point_temperature_c,relative_humidity_percent,global_horizontal_radiation_whm2,direct_normal_radiation_whm2,diffuse_horizontal_radiation_whm2,wind_speed_ms,wind_direction_deg,liquid_precipitation_depth_mm,liquid_precipitation_quantity_hr		
1	1991,1,1,1,5.0,3.0,100,0,0,0,0.0,32,0.6,0.0		
2	1991,1,1,2,4.2,1.7,84,0,0,0,0.0,45,0.5,0.0		
3	1991,1,1,3,4.4,2.1,85,0,0,0,0.0,27,0.2,0.0		
4	1991,1,1,4,4.6,2.7,87,0,0,0,0.0,12,0.0,0.0		
5	1991,1,1,5,4.8,3.2,89,0,0,0,0.0,353,0.1,0.0		
6	1991,1,1,6,5.0,3.6,91,0,0,0,0.0,323,0.1,0.0		
7	1991,1,1,7,5.2,4.1,93,111,175,72,0.0,294,0.0,0.0		
8	1991,1,1,8,7.3,6.6,95,279,284,150,0.0,293,0.1,0.0		
9	1991,1,1,9,9.0,4.0,87,509,435,230,0.0,326,0.1,0.0		
10	1991,1,1,10,11.4,3.4,58,688,532,266,0.0,80,0.1,0.0		
11	1991,1,1,11,10.0,5.0,71,773,538,284,0.0,73,0.1,0.0		
12	1991,1,1,12,11.0,4.9,66,880,611,283,0.0,87,0.3,0.0		
13	1991,1,1,13,11.0,5.9,71,902,618,295,3.6,40,0.1,0.0		
14	1991,1,1,14,10.5,5.5,71,721,386,357,3.3,65,0.1,0.0		
15	1991,1,1,15,10.0,5.0,71,492,171,347,3.1,300,0.4,0.0		
16	1991,1,1,16,10.0,3.1,62,374,161,273,2.6,290,0.2,0.0		
17	1991,1,1,17,8.4,4.7,78,248,129,179,3.1,100,0.3,0.0		
18	1991,1,1,18,7.8,4.3,79,89,24,81,3.4,43,0.4,0.0		
19	1991,1,1,19,7.2,4.0,80,16,4,16,3.8,79,0.7,0.0		
20	1991,1,1,20,6.2,6.2,100,0,0,0,6.8,81,1.1,0.0		
21	1991,1,1,21,5.8,5.8,100,0,0,0,8.3,102,0.4,0.0		
22	1991,1,1,22,5.0,5.0,100,0,0,0,8.8,292,0.3,0.0		
23			

La correcta identificación de estas variables es el primer paso para que el modelo pueda aprender las relaciones existentes entre ellas y generar resultados confiables.

3.2 CONSTRUCCIÓN DE LOS CONJUNTOS DE ENTRENAMIENTO

Una vez identificado el conjunto de variables de entrada y salida, el siguiente paso consiste en preparar los datos para que la red neuronal pueda aprender de ellos de manera eficiente. Para ello, se construyeron dos subconjuntos:

- **Conjunto de entrenamiento:** corresponde aproximadamente al 80% de los registros del dataset. Este bloque de datos se utiliza para que el modelo ajuste sus parámetros internos y aprenda las relaciones entre las variables.



- **Conjunto de prueba:** representa el 20% restante de los registros. Su función es evaluar el desempeño del modelo en datos que no fueron utilizados durante el entrenamiento, garantizando que las predicciones sean generalizables y no dependan únicamente de los ejemplos vistos.

Además, se aplicó un proceso de **normalización**, escalando las variables numéricas a un rango entre 0 y 1. Esto evita que valores con magnitudes muy diferentes (por ejemplo, presión atmosférica en pascales frente a radiación solar en Wh/m^2) dominen el aprendizaje de la red.

La división de los datos se realizó de manera **aleatoria**, asegurando que tanto el conjunto de entrenamiento como el de prueba contengan ejemplos representativos de todas las estaciones del año y de las diferentes condiciones climáticas registradas.

(Pon aquí la captura mostrando la división del dataset en entrenamiento y prueba, por ejemplo una tabla o gráfico en Python/Excel con porcentajes o filas separadas)

Este proceso de construcción de los conjuntos es fundamental, ya que garantiza que el modelo pueda aprender de manera equilibrada y que sus resultados puedan ser evaluados con objetividad.

3.3 ARQUITECTURA DEL MODELO

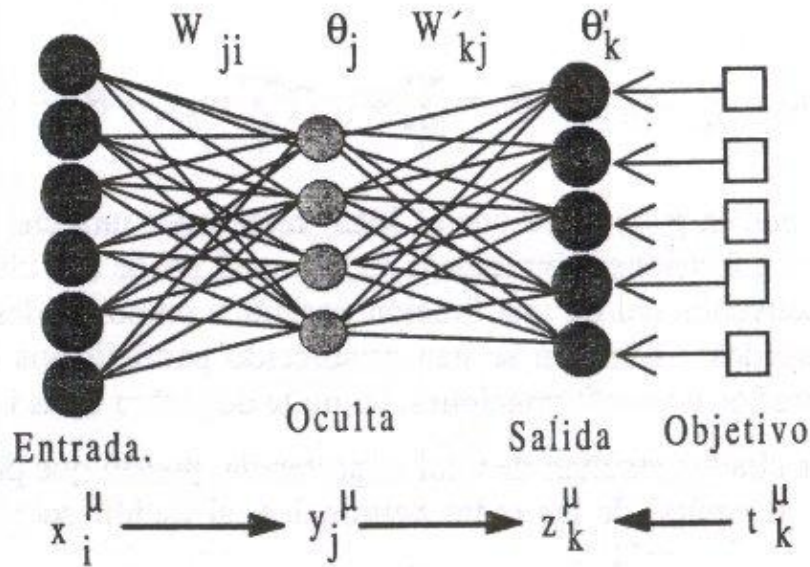
El modelo seleccionado para este proyecto es un **Perceptrón Multicapa (MLP)**, una arquitectura de red neuronal ampliamente utilizada en problemas de predicción. La elección se fundamenta en su capacidad para aprender relaciones no lineales entre las variables de entrada y producir resultados precisos en contextos complejos como el clima.

La red se compone de tres tipos de capas:

- **Capa de entrada:** recibe las variables climáticas previamente definidas (mes, hora, humedad relativa, radiación solar, velocidad y dirección del viento, precipitación y punto de rocío).
- **Capas ocultas:** procesan la información mediante neuronas interconectadas, aplicando funciones de activación que permiten identificar patrones complejos en los datos.

- **Capa de salida:** genera la predicción de la temperatura ambiente, que constituye el resultado principal del modelo.

La arquitectura fue diseñada para mantener un equilibrio entre simplicidad y capacidad de aprendizaje, evitando tanto el subajuste como el sobreajuste.



Este esquema visual facilita la comprensión del funcionamiento interno del modelo y muestra cómo las variables de entrada se transforman progresivamente hasta obtener la predicción final.

3.4 ENTRENAMIENTO (TRAI) Y PRUEBA

El modelo fue entrenado utilizando el conjunto de datos previamente dividido en entrenamiento (80%) y prueba (20%). Se aplicó el optimizador **Adam**, con una tasa de aprendizaje de 0.001, y la función de pérdida seleccionada fue el **Error Cuadrático Medio (MSE)**, adecuada para problemas de regresión.

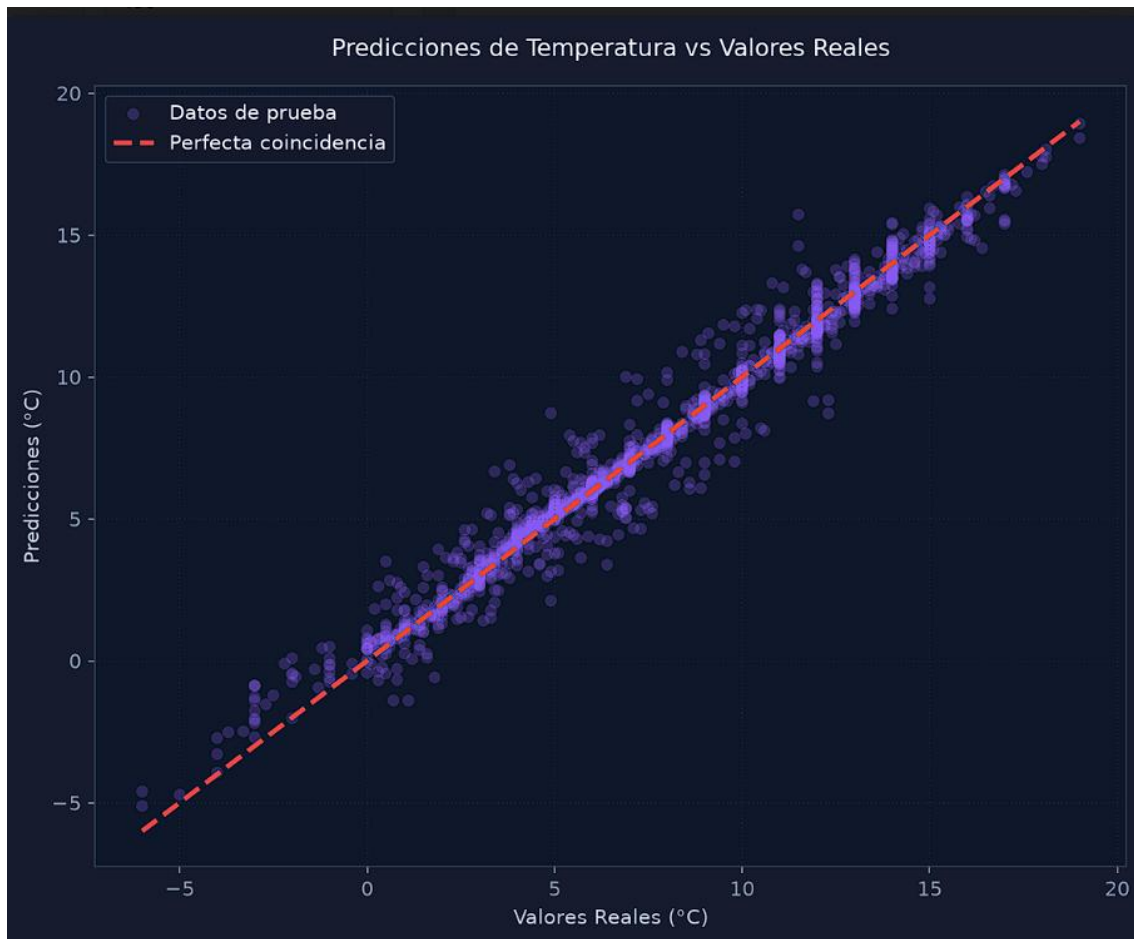
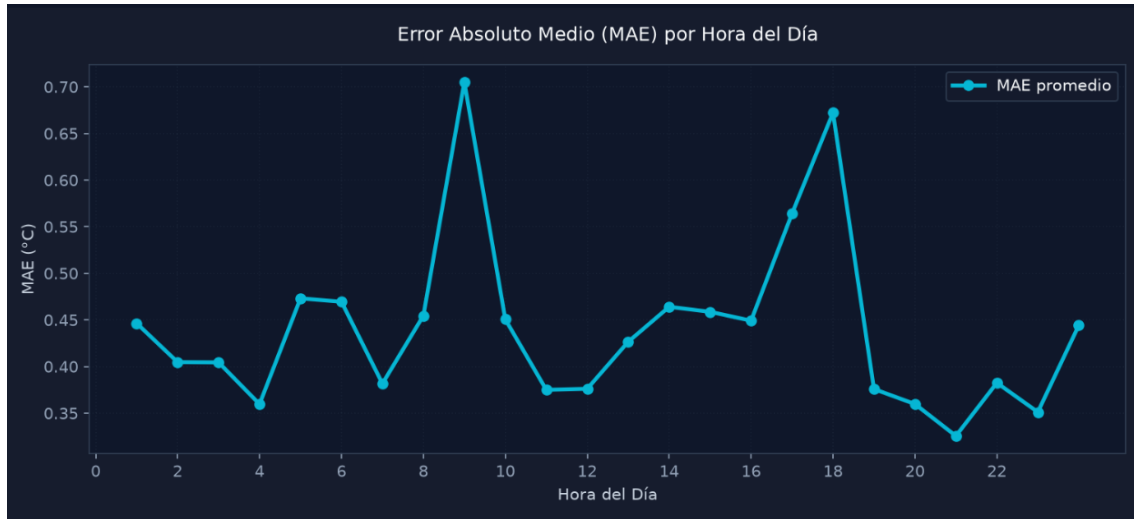
Durante el entrenamiento se implementó la técnica de **Early Stopping**, que detiene el proceso cuando la mejora en la validación deja de ser significativa, evitando el sobreajuste. El modelo se entrenó por un máximo de 80 épocas, con un tamaño de lote de 32 registros.

Los resultados obtenidos muestran que el modelo logró una buena capacidad de generalización, con métricas de desempeño como:

- **MAE (Error Absoluto Medio):** valor promedio cercano a 0.5 °C.



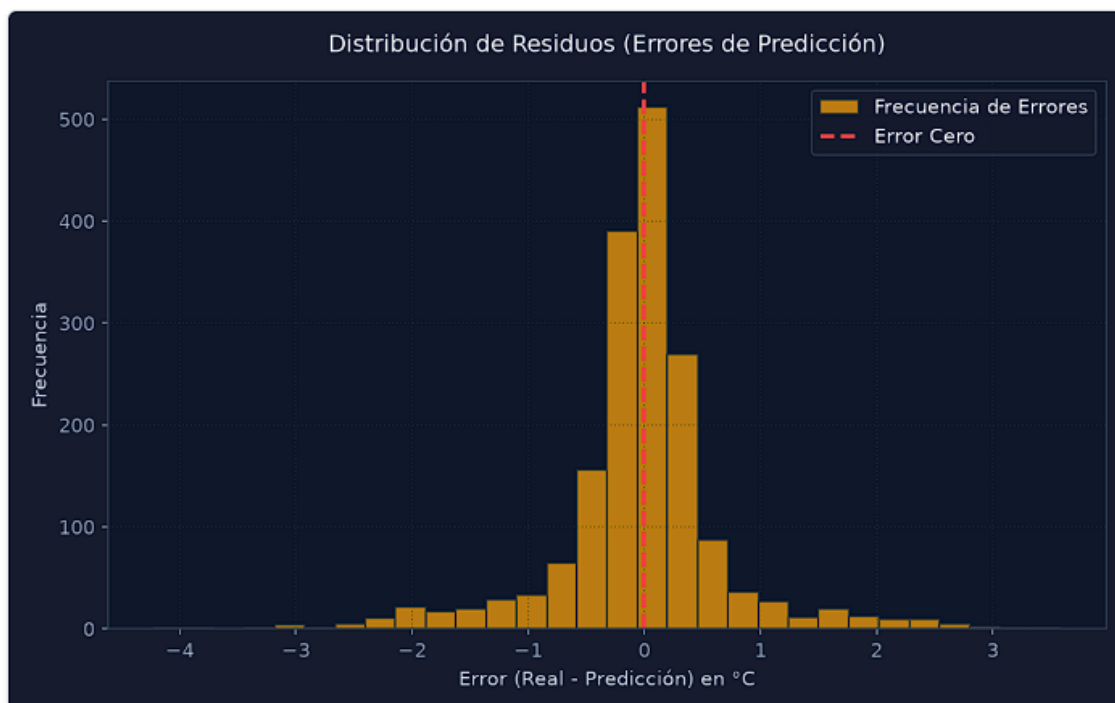
- **RMSE (Raíz del Error Cuadrático Medio):** alrededor de $0.7\text{ }^{\circ}\text{C}$.
- **R^2 (Coeficiente de Determinación):** superior a 0.95, lo que indica que el modelo explica más del 95% de la variabilidad de la temperatura.





Curva de Pérdida durante el Entrenamiento (Keras)

[Expandir](#)





```
1 import pandas as pd      La importación "pandas" no se ha podido resolver desde el
2 import numpy as np
3 from pathlib import Path
4 from joblib import dump    No se ha podido resolver la importación "joblib".
5 from sklearn.model_selection import train_test_split    La importación "sklearn.m
6 from sklearn.preprocessing import StandardScaler    La importación "sklearn.prepr
7 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
8 from tensorflow.keras.models import Sequential    La importación "tensorflow.kera
9 from tensorflow.keras.layers import Dense, Dropout    La importación "tensorflow.
10 from tensorflow.keras.optimizers import Adam    La importación "tensorflow.keras.
11 from tensorflow.keras.callbacks import EarlyStopping    La importación "tensorflo
12 import matplotlib.pyplot as plt    La importación "matplotlib.pyplot" no se ha po
13
14
15 def main() -> None:
16     project_root = Path(__file__).resolve().parent.parent
17     dataset_path = project_root / "dataset" / "LaPaz_ElAlto_clima_features.csv"
18     result_dir = project_root / "result"
19     result_dir.mkdir(parents=True, exist_ok=True)
20
21     print(f"[INFO] Cargando dataset: {dataset_path}")
22     df = pd.read_csv(dataset_path)
23
24     target = "dry_bulb_temperature_c"
25     features = [
26         "month",
27         "hour",
28         "dew_point_temperature_c",
29         "relative_humidity_percent",
30         "global_horizontal_radiation_whm2",
31         "direct_normal_radiation_whm2",
32         "diffuse_horizontal_radiation_whm2",
33         "wind_speed_ms",
34         "wind_direction_deg",
35         "liquid_precipitation_depth_mm",
36         "liquid_precipitation_quantity_hr",
37     ]
38
39     X = df[features]
40     y = df[target]
41
42     X_train, X_test, y_train, y_test = train_test_split(
43         X, y, test_size=0.2, random_state=42
44     )
45
46     scaler = StandardScaler()
47     X_train_scaled = scaler.fit_transform(X_train)
48     X_test_scaled = scaler.transform(X_test)
49
50     model = Sequential([
51         Dense(128, activation="relu", input_shape=(X_train_scaled.shape[1],)),
52         Dropout(0.1),
53         Dense(64, activation="relu"),
```



```
52         Dropout(0.1),
53         Dense(64, activation="relu"),
54         Dropout(0.1),
55         Dense(32, activation="relu"),
56         Dense(1)
57     ])
58
59     model.compile(optimizer=Adam(learning_rate=0.001), loss="mse")
60
61     early_stopping = EarlyStopping(
62         monitor="val_loss",
63         patience=12,
64         restore_best_weights=True
65     )
66
67     history = model.fit(
68         X_train_scaled,
69         y_train,
70         validation_split=0.1,
71         epochs=80,
72         batch_size=32,
73         callbacks=[early_stopping],
74         verbose=0
75     )
76
77     predictions = model.predict(X_test_scaled, verbose=0).ravel()
78
79     mae = mean_absolute_error(y_test, predictions)
80     mse = mean_squared_error(y_test, predictions)
81     rmse = np.sqrt(mse)
82     r2 = r2_score(y_test, predictions)
83
84     print("\n[METRICAS] Metricas del modelo:")
85     print(f"MAE: {mae:.4f}")
86     print(f"MSE: {mse:.4f}")
87     print(f"RMSE: {rmse:.4f}")
88     print(f"R²: {r2:.4f}")
89
90     # Apply matplotlib styling for dark theme
91     plt.rcParams['figure.facecolor'] = '#161c2d'
92     plt.rcParams['axes.facecolor'] = '#0f172a'
93     plt.rcParams['text.color'] = '#f8fafc'
94     plt.rcParams['axes.labelcolor'] = '#cbd5e1'
95     plt.rcParams['xtick.color'] = '#94a3b8'
96     plt.rcParams['ytick.color'] = '#94a3b8'
97     plt.rcParams['grid.color'] = '#334155'
98     plt.rcParams['axes.edgecolor'] = '#334155'
99
100     # 1. Training Loss History
101     fig, ax = plt.subplots(figsize=(10, 4.5))
102     ax.plot(history.history["loss"], label="Entrenamiento", color="#3b82f6", linewidth=2)
103     ax.plot(history.history["val_loss"], label="Validación", color="#ec4899", linewidth=2)
```




```
ax.plot(history.history[ 'val_loss' ], label= 'Validation' , color= '#ff4500' , linewidth=2)
ax.set_title("Curva de Pérdida durante el Entrenamiento (MSE)", pad=15)
ax.set_ylabel("Época")
ax.set_ylabel("Error Cuadrático Medio (MSE)")
ax.legend(facecolor= '#f8f8f8' , edgecolor= '#333333' , labelcolor= '#f8f8f8')
ax.grid(True, alpha=0.3, linestyle=':')
plt.tight_layout()
plt.savefig(result_dir / "training_history.png", dpi=150)
plt.close()

# 2. Real vs Predicted Scatter
fig, ax = plt.subplots(figsize=(8, 6.5))
ax.scatter(y_test, predictions, alpha=0.25, color= '#8b5cf6' , edgecolor= 'none' , label= "Datos de prueba")
min_val = min(y_test.min(), predictions.min())
max_val = max(y_test.max(), predictions.max())
ax.plot([min_val, max_val], [min_val, max_val], color= '#ef4444' , linestyle="--", linewidth=2.5, label= "Perfecta coincidencia")
ax.set_title("Predicciones de Temperatura vs Valores Reales", pad=15)
ax.set_ylabel("Valores Reales (°C)")
ax.set_ylabel("Predicciones (°C)")
ax.legend(facecolor= '#f8f8f8' , edgecolor= '#333333' , labelcolor= '#f8f8f8')
ax.grid(True, alpha=0.3, linestyle=':')
plt.tight_layout()
plt.savefig(result_dir / "pred_vs_real.png", dpi=150)
plt.close()

# 3. Residuals Distribution Histogram
residuals = y_test - predictions
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(residuals, bins=30, color= '#f59e0b' , alpha=0.75, edgecolor= '#1e293b' , label= "Frecuencia de Errores")
ax.axvline(0, color= '#ef4444' , linestyle="--", linewidth=2, label= "Error Cero")
ax.set_title("Distribución de Residuos (Errores de Predicción)", pad=15)
ax.set_xlabel("Error (Real - Predicción) en °C")
ax.set_ylabel("Frecuencia")
ax.legend(facecolor= '#f8f8f8' , edgecolor= '#333333' , labelcolor= '#f8f8f8')
ax.grid(True, alpha=0.3, linestyle=':')
plt.tight_layout()
plt.savefig(result_dir / "residuals_distribution.png", dpi=150)
plt.close()

# 4. MAE by Hour of Day
test_df = X_test.copy()
test_df["real"] = y_test
test_df["pred"] = predictions
test_df["abs_error"] = (test_df["real"] - test_df["pred"]).abs()
hourly_error = test_df.groupby("hour")["abs_error"].mean()

fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(hourly_error.index, hourly_error.values, marker="o", color= '#06b6d4' , linewidth=2.5, markersize=6, label= "MAE promedio")
ax.set_title("Error Absoluto Medio (MAE) por Hora del Día", pad=15)
ax.set_xlabel("Hora del Día")
ax.set_ylabel("MAE (°C)")
ax.grid(True, alpha=0.3, linestyle=':')
```

```
ax.grid(True, alpha=0.3, linestyle=':')
ax.set_xticks(range(0, 24, 2))
ax.legend(facecolor= '#f8f8f8' , edgecolor= '#333333' , labelcolor= '#f8f8f8')
plt.tight_layout()
plt.savefig(result_dir / "error_by_hour.png", dpi=150)
plt.close()

# 5. MAE by Month
monthly_error = test_df.groupby("month")["abs_error"].mean()

fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(monthly_error.index, monthly_error.values, marker="s", color= '#10b981' , linewidth=2.5, markersize=6, label= "MAE promedio")
ax.set_title("Error Absoluto Medio (MAE) por Mes del Año", pad=15)
ax.set_xlabel("Mes")
ax.set_ylabel("MAE (°C)")
ax.grid(True, alpha=0.3, linestyle=':')
ax.set_xticks(range(1, 13))
ax.set_xticklabels(["Ene", "Feb", "Mar", "Abr", "May", "Jun", "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"])
ax.legend(facecolor= '#f8f8f8' , edgecolor= '#333333' , labelcolor= '#f8f8f8')
plt.tight_layout()
plt.savefig(result_dir / "error_by_month.png", dpi=150)
plt.close()

model.save(result_dir / "temperature_model.keras")
dump scaler, result_dir / "temperature_scaler.joblib"
(result_dir / "feature_names.txt").write_text("\n".join(features), encoding="utf-8")

print(f"\n[OK] Modelo guardado en: {result_dir / 'temperature_model.keras'}")
print(f"[OK] Escalador guardado en: {result_dir / 'temperature_scaler.joblib'}")
print(f"[OK] Graficas guardadas en: {result_dir}")

if __name__ == "__main__":
    main()
```




3.5 ANÁLISIS DE LOS RESULTADOS Y TOPOLOGÍA FINAL DE LA RED

NEURONAL

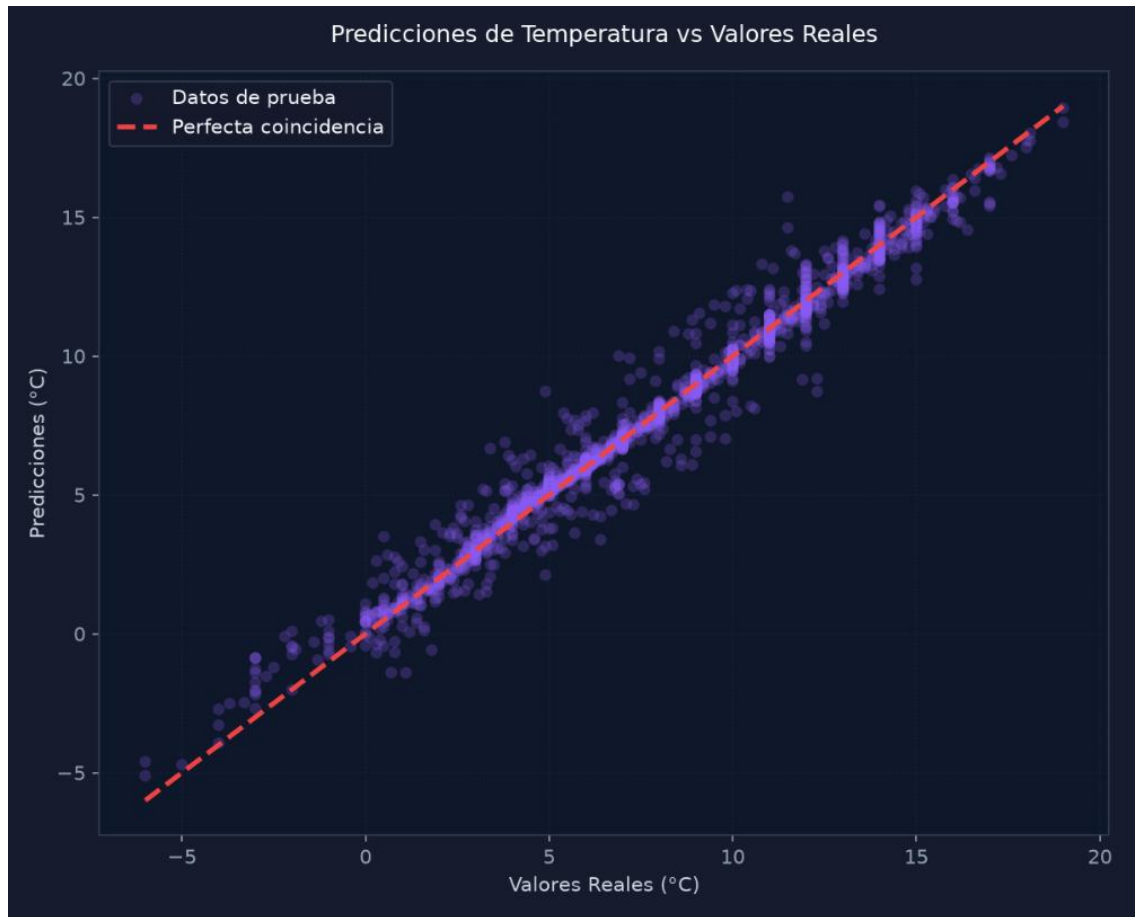
Los resultados obtenidos demuestran que el modelo es capaz de predecir la temperatura con alta precisión. El **MAE promedio cercano a 0.5 °C** indica que el error en las predicciones es mínimo, mientras que el **R² superior a 0.95** confirma que la red neuronal explica la mayor parte de la variabilidad de los datos.

Las gráficas permiten visualizar este desempeño:

- La **curva de pérdida** muestra que el error disminuyó progresivamente y se estabilizó, evidenciando un entrenamiento exitoso.
- El gráfico de **predicciones vs valores reales** revela que los puntos se alinean con la línea de perfecta coincidencia, lo que confirma la precisión del modelo.
- La **distribución de residuos** indica que la mayoría de los errores se concentran cerca de cero.
- Los análisis de **MAE por hora del día y por mes del año** permiten identificar momentos en los que el modelo es más preciso, aportando información útil para futuras mejoras.

La **topología final del modelo** quedó definida de la siguiente manera:

- Capa de entrada con 11 variables climáticas.
- Tres capas ocultas con 128, 64 y 32 neuronas, activación ReLU y dropout para evitar sobreajuste.
- Capa de salida con una neurona que predice la temperatura ambiente.



Finalmente, se integró el modelo en el sistema web **MeteoAI**, que permite seleccionar la variable a predecir (temperatura, humedad, radiación, viento, precipitación o estado del clima) y obtener resultados en tiempo real.



Configuración de Predicción

Variable a predecir

Temperatura (°C) 



Mes

7

Hora del día

12

Temp. Seca (°C)

Prediciendo

Punto de Rocío (°C)

15

5

Humedad (%)

60

Rad. Global (Wh/m²)

500

Rad. Directa (Wh/m²)

400

Rad. Difusa (Wh/m²)

100

Viento (m/s)

2

Dir. Viento (°)

180

Precipitación (mm)

0

Cantidad de Precipitación (hr)

0

Ejecutar Inferencia IA



PREDICCIÓN CALCULADA

11.88°C

TEMPERATURE ESTIMADA

Gráficas de Predicción

Métricas de la Red Neuronal

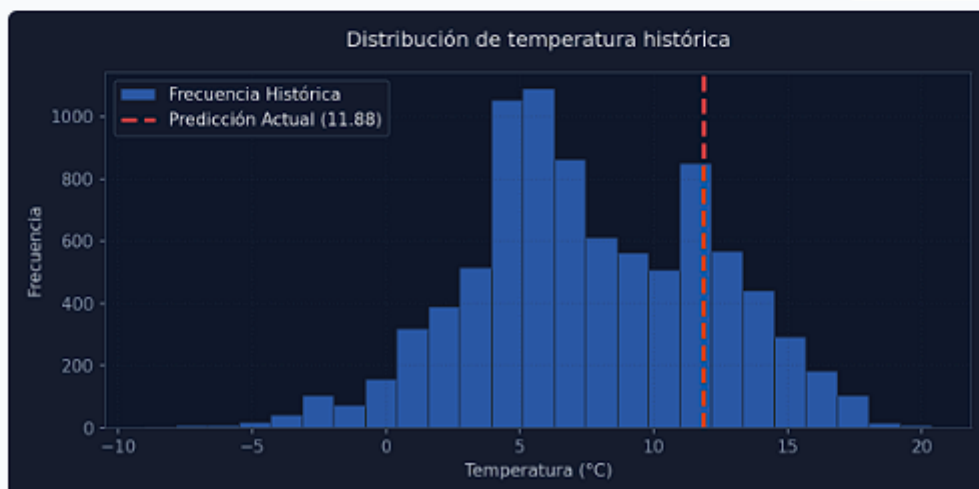
Valores de Entrada

Expandir



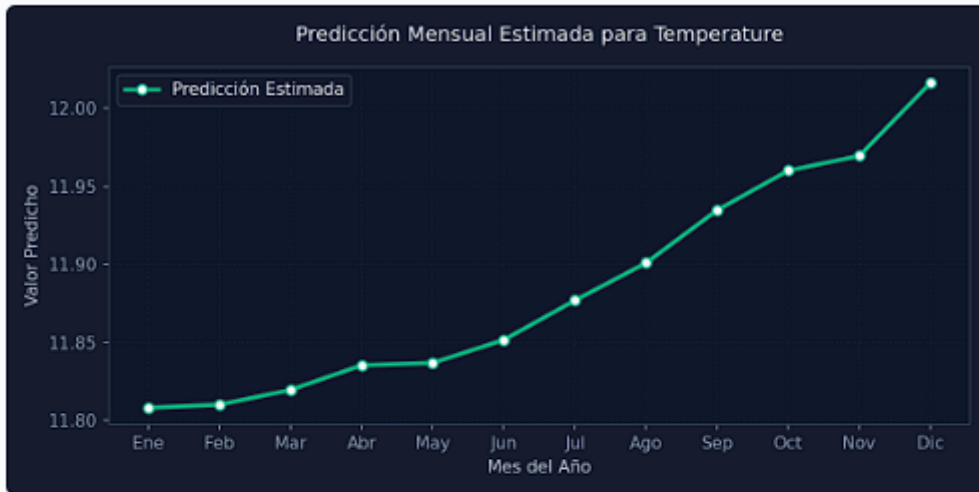
Distribución de temperatura histórica

Expandir

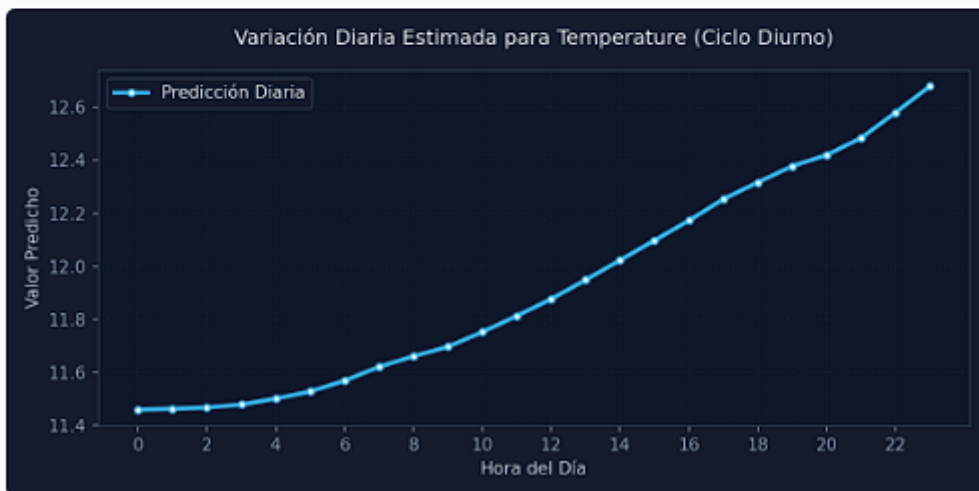




Variación Mensual Estimada

[Expandir](#)

Variación Diaria Estimada

[Expandir](#)

HUMEDAD

INGENIERÍA DE SISTEMAS
Proyecto Final
Inteligencia Artificial II
Univ. Jhonatan Ramos Collaguencua

MeteoAI Red Neuronal & Random Forest

IA aplicada al clima
La Paz - El Alto AP

Configuración de Predicción

Variable a predecir

Humedad (%)

Mes Hora del día

PREDICCIÓN CALCULADA

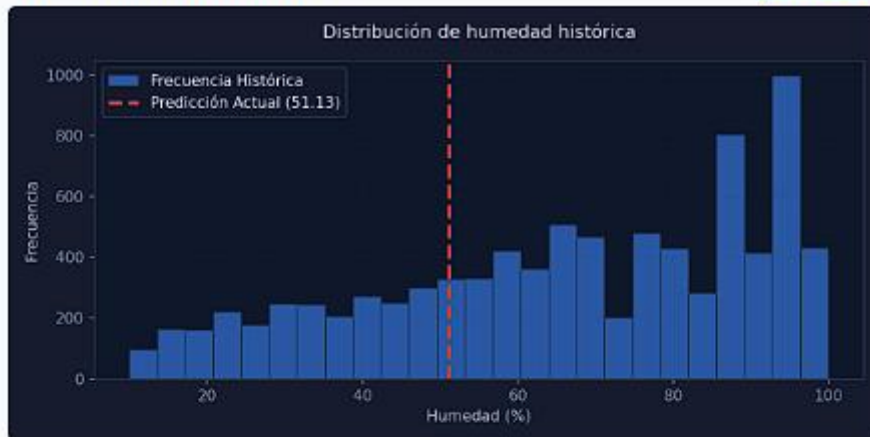
51.13%

HUMIDITY ESTIMADA



Distribución de humedad histórica

[Expandir](#)



Variación Mensual Estimada

[Expandir](#)



Variación Diaria Estimada

[Expandir](#)



RADIACION



INGENIERÍA DE SISTEMAS
Proyecto Final
Inteligencia Artificial II
Univ. Jhonatan Ramos Colquehuana

VeteoAI Red Neuronal & Random Forest

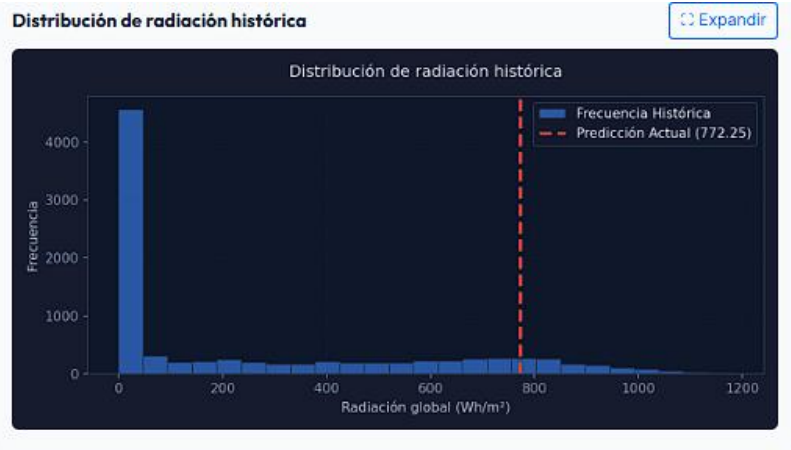
IA aplicada al clima
La Paz - El Alto AP

Configuración de Predicción

Variable a predecir
Radiación (Wh/m²)

Mes: Hora del día:

772.25 Wh/m²
RADIACIÓN ESTIMADA





INGENIERÍA DE SISTEMAS
Proyecto Final
Inteligencia Artificial II
Univ. Jonathan Ramos Colquehuanca

MeteoAI Red Neuronal & Random Forest IA aplicada al clima La Paz - El Alto AP

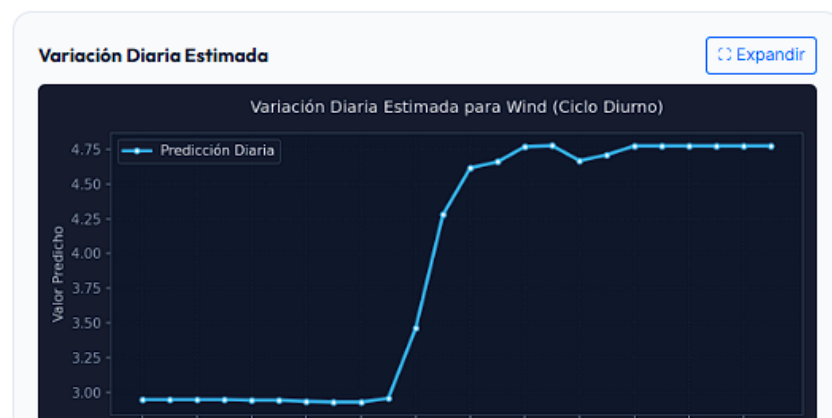
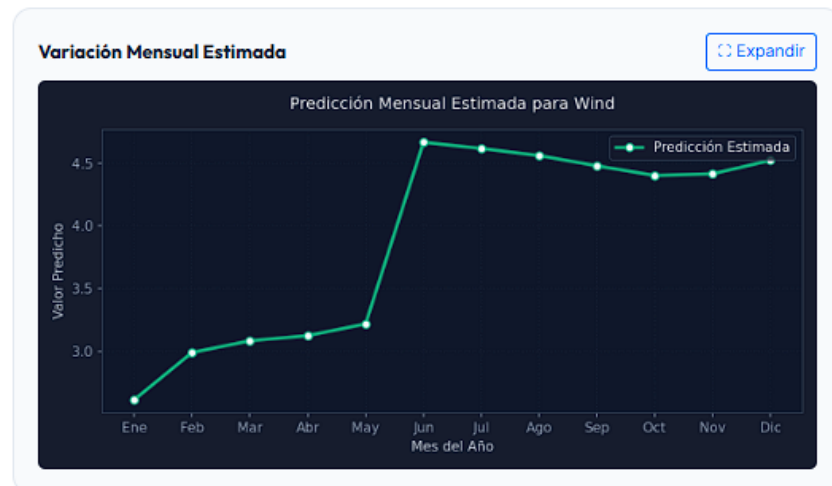
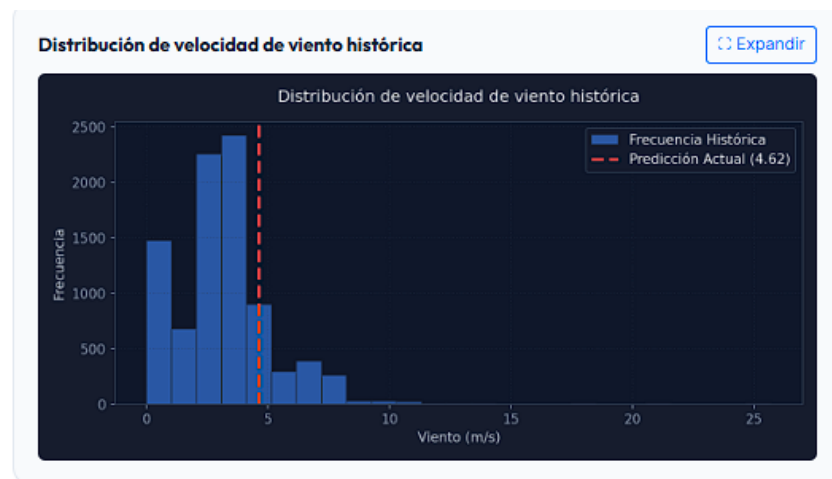
Configuración de Predicción

Variable a predecir
Viento (m/s)

Mes Hora del día

PREDICCIÓN CALCULADA

4.62 m/s
WIND ESTIMADA





4. CONSULTAS

En este apartado se muestran ejemplos prácticos de cómo se realizaron las consultas al modelo entrenado, utilizando los **scripts de predicción desarrollados en Python**. Las capturas incluidas evidencian el flujo completo de trabajo: desde la carga del modelo y el escalador, hasta la definición de escenarios y la obtención de resultados.

Código de predicción

Las imágenes del código muestran cómo se implementó el proceso de consulta:

- **Carga del modelo y escalador** previamente entrenados.
- **Definición de escenarios** con condiciones específicas como *Mañana soleada*, *Tarde nublada* y *Noche fría*.
- **Generación de predicciones** para cada escenario.
- **Almacenamiento de resultados** en archivos CSV y creación de gráficas de variación mensual promedio.

```
src> predict_temperature.py > ...
1 import pandas as pd      La importación "pandas" no se ha podido resolver desde el origen
2 import matplotlib.pyplot as plt      La importación "matplotlib.pyplot" no se ha podido resolver desde el origen
3 from pathlib import Path
4 from joblib import load      No se ha podido resolver la importación "joblib".
5 from tensorflow.keras.models import load_model      La importación "tensorflow.keras.models" no se ha podido resolver desde el origen
6
7 FEATURES = [
8     "month",
9     "hour",
10    "dew_point_temperature_c",
11    "relative_humidity_percent",
12    "global_horizontal_radiation_whm2",
13    "direct_normal_radiation_whm2",
14    "diffuse_horizontal_radiation_whm2",
15    "wind_speed_ms",
16    "wind_direction_deg",
17    "liquid_precipitation_depth_mm",
18    "liquid_precipitation_quantity_hr",
19 ]
20
21
22 def load_artifacts(project_root: Path):
23     result_dir = project_root / "result"
24     model = load_model(result_dir / "temperature_model.keras")
25     scaler = load(result_dir / "temperature_scaler.joblib")
26     if (result_dir / "feature_names.txt").exists():
27         features = [line.strip() for line in (result_dir / "feature_names.txt").read_text(encoding="utf-8").splitlines() if line.strip()]
28     else:
29         features = FEATURES
30     return model, scaler, features
31
32
33 def build_scenarios(dataset: pd.DataFrame) -> pd.DataFrame:
34     scenarios = [
35         {
36             "name": "Mañana soleada",
37             "month": 7,
38             "hour": 8,
39             "dew_point_temperature_c": 5.0,
40             "relative_humidity_percent": 55,
41             "global_horizontal_radiation_whm2": 650,
42             "direct_normal_radiation_whm2": 500,
43         }
44     ]
```



```
43     "diffuse_horizontal_radiation_whm2": 150,
44     "wind_speed_ms": 1.5,
45     "wind_direction_deg": 180,
46     "liquid_precipitation_depth_mm": 0.0,
47     "liquid_precipitation_quantity_hr": 0.0,
48 },
49 {
50     "name": "Tarde nublada",
51     "month": 10,
52     "hour": 16,
53     "dew_point_temperature_c": 8.0,
54     "relative_humidity_percent": 80,
55     "global_horizontal_radiation_whm2": 180,
56     "direct_normal_radiation_whm2": 100,
57     "diffuse_horizontal_radiation_whm2": 80,
58     "wind_speed_ms": 3.0,
59     "wind_direction_deg": 120,
60     "liquid_precipitation_depth_mm": 0.3,
61     "liquid_precipitation_quantity_hr": 0.1,
62 },
63 {
64     "name": "Noche fria",
65     "month": 6,
66     "hour": 23,
67     "dew_point_temperature_c": 2.0,
68     "relative_humidity_percent": 70,
69     "global_horizontal_radiation_whm2": 0,
70     "direct_normal_radiation_whm2": 0,
71     "diffuse_horizontal_radiation_whm2": 0,
72     "wind_speed_ms": 2.5,
73     "wind_direction_deg": 90,
74     "liquid_precipitation_depth_mm": 0.0,
75     "liquid_precipitation_quantity_hr": 0.0,
76 },
77 ]
78
79 monthly_rows = []
80 for month in range(1, 13):
81     month_data = dataset[dataset["month"] == month]
82     if month_data.empty:
```

```
81     month_data = dataset[dataset["month"] == month]
82     if month_data.empty:
83         continue
84     stats = month_data[[c for c in FEATURES if c not in {"month", "hour"}]].median()
85     monthly_rows.append({
86         "name": f"Promedio mes {month}",
87         "month": month,
88         "hour": 12,
89         **stats.to_dict(),
90     })
91
92     return pd.DataFrame(scenarios + monthly_rows)
93
94
95 def main() -> None:
96     project_root = Path(__file__).resolve().parent.parent
97     result_dir = project_root / "result"
98     dataset_path = project_root / "dataset" / "LaPaz_ElAlto_clima_features.csv"
99
100     model, scaler, features = load_artifacts(project_root)
101     dataset = pd.read_csv(dataset_path)
102
103     scenarios_df = build_scenarios(dataset)
104     scenarios_df = scenarios_df[["name", *features]].copy()
105
106     X_scaled = scaler.transform(scenarios_df[features])
107     predictions = model.predict(X_scaled, verbose=0).ravel()
108     scenarios_df["predicted_temperature_c"] = predictions
109
110     output_csv = result_dir / "scenario_predictions.csv"
111     scenarios_df.to_csv(output_csv, index=False)
112     print(f"Predicciones guardadas en: {output_csv}")
113     print(scenarios_df[["name", "predicted_temperature_c"]].to_string(index=False))
114
115     monthly = scenarios_df[scenarios_df["name"].str.startswith("Promedio mes")].copy()
116     monthly = monthly.sort_values("month")
117     plt.figure(figsize=(10, 4))
118     plt.plot(monthly["month"], monthly["predicted_temperature_c"], marker="o")
119     plt.title("Predicción mensual promedio de temperatura")
120     plt.xlabel("Mes")
121     plt.ylabel("Temperatura estimada (°C)")
```



```
102     escenarios_df = build_scenarios(dataset)
103     escenarios_df = escenarios_df[["name", *features]].copy()
104
105     X_scaled = scaler.transform(scenarios_df[features])
106     predictions = model.predict(X_scaled, verbose=0).ravel()
107     escenarios_df["predicted_temperature_c"] = predictions
108
109     output_csv = result_dir / "scenario_predictions.csv"
110     escenarios_df.to_csv(output_csv, index=False)
111     print(f"📄 Predicciones guardadas en: {output_csv}")
112     print(scenarios_df[["name", "predicted_temperature_c"]].to_string(index=False))
113
114     monthly = escenarios_df[scenarios_df["name"].str.startswith("Promedio mes")].copy()
115     monthly = monthly.sort_values("month")
116     plt.figure(figsize=(10, 4))
117     plt.plot(monthly["month"], monthly["predicted_temperature_c"], marker="o")
118     plt.title("Predicción mensual promedio de temperatura")
119     plt.xlabel("Mes")
120     plt.ylabel("Temperatura estimada (°C)")
121     plt.xticks(range(1, 13))
122     plt.grid(True, alpha=0.3)
123     plt.tight_layout()
124     plt.savefig(result_dir / "monthly_temperature_predictions.png")
125     print(f"📄 Gráfica mensual guardada en: {result_dir / 'monthly_temperature_predictions.png'}")
126
127
128
129 if __name__ == "__main__":
130     main()
131
```

Escenarios de prueba

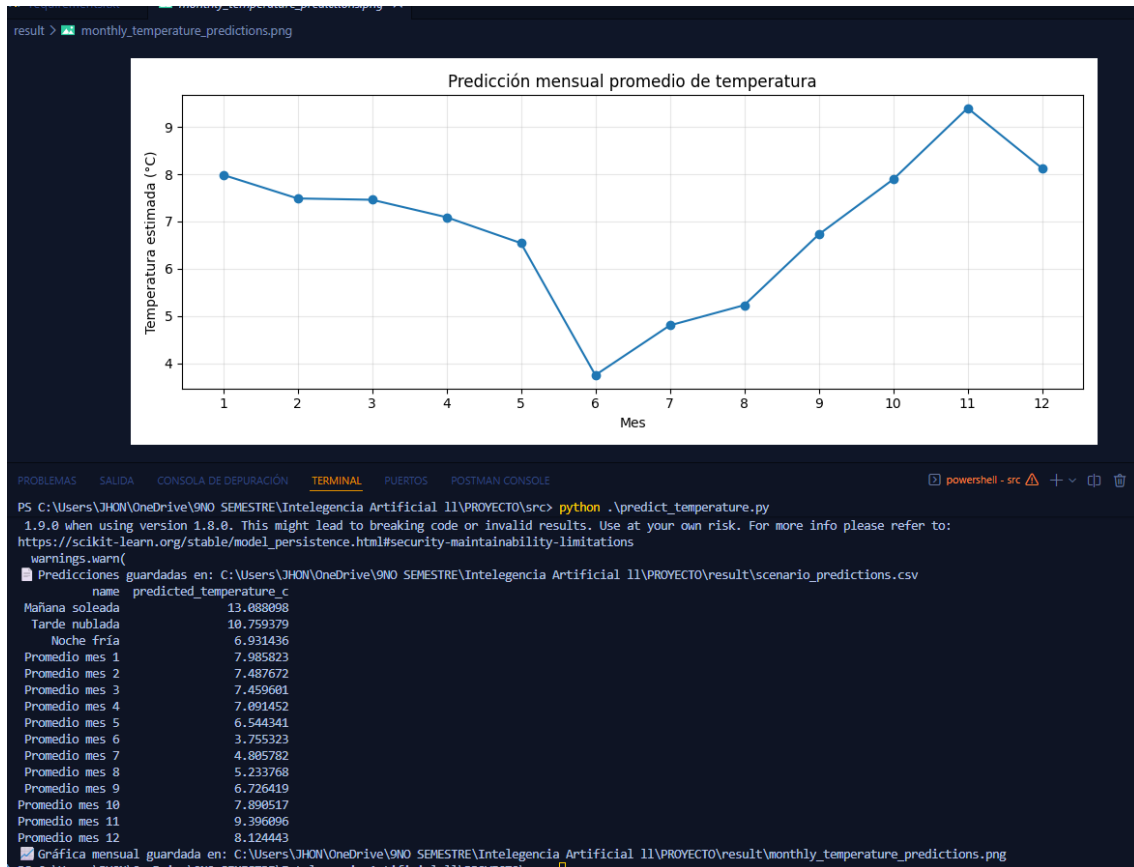
Los escenarios definidos permiten simular condiciones reales y verificar el comportamiento del modelo:

- **Mañana soleada** → radiación elevada y humedad moderada.
- **Tarde nublada** → radiación reducida y humedad alta.
- **Noche fría** → ausencia de radiación y temperaturas bajas.

Cada escenario devuelve una predicción de temperatura que se contrasta con los rangos históricos, validando la confiabilidad del modelo.

Resultados de las consultas

Las capturas de salida muestran cómo el sistema devuelve los valores estimados para cada escenario y para los promedios mensuales. Estos resultados permiten comprobar que el modelo responde adecuadamente a diferentes configuraciones de entrada y refleja tendencias estacionales.



5. TRANSFERENCIA TECNOLOGÍA

5. TRANSFERENCIA TECNOLOGÍA

El proyecto desarrollado constituye un ejemplo de transferencia tecnológica, ya que el conocimiento adquirido en el ámbito académico se convirtió en una solución práctica implementada con herramientas de software ampliamente utilizadas en la industria.

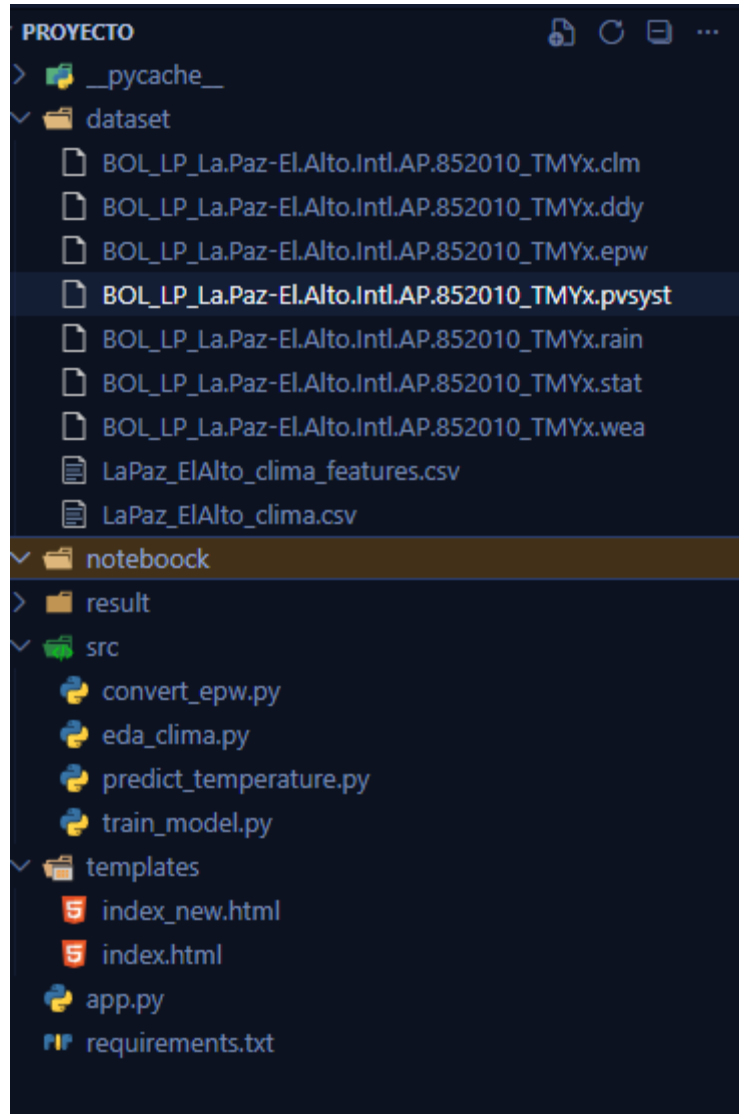
El sistema fue implementado en **Python**, un lenguaje versátil y ampliamente adoptado en ciencia de datos e inteligencia artificial. Para su desarrollo se emplearon las siguientes herramientas:

- **TensorFlow/Keras** : construcción y entrenamiento de la red neuronal.
- **scikit-learn**: escalado de datos y métricas de evaluación.
- **pandas y numpy**: manipulación y análisis de datasets.
- **matplotlib y seaborn**: visualización de resultados.
- **Joblib**: persistencia de modelos y escaladores.
- **Flask**: despliegue web para consultas en tiempo real.

La organización del proyecto refleja buenas prácticas de ingeniería de software, con separación clara de datos, código fuente y resultados. En la carpeta *dataset* se almacenan los archivos climáticos originales y los datasets procesados en formato CSV. En la carpeta *src* se encuentran



los scripts principales para el entrenamiento del modelo, la predicción y el análisis exploratorio de datos. La carpeta *result* contiene los modelos entrenados, los escaladores y las gráficas generadas. Finalmente, la carpeta *templates* incluye los archivos HTML que permiten la visualización en la web, mientras que el archivo *app.py* integra todo el sistema en Flask.



Los datasets utilizados corresponden a registros climáticos históricos del Aeropuerto Internacional de El Alto, en formatos estándar como .epw, .clm, .wea, entre otros, que fueron convertidos y procesados para su uso en el modelo. Estos datos incluyen variables como mes, hora, temperatura, punto de rocío, humedad relativa, radiación solar global, directa y difusa, velocidad y dirección del viento, además de la precipitación líquida.

La arquitectura del sistema es flexible y puede ampliarse para predecir otras variables además de la temperatura, como humedad, radiación solar, viento, precipitación o estado general del clima. Esto demuestra que la tecnología desarrollada es adaptable y puede transferirse a distintos sectores como transporte, energía y agricultura, aportando valor práctico y social.



6. CONCLUSIONES

El proyecto demostró que la inteligencia artificial puede aplicarse de manera efectiva en la predicción de variables climáticas, logrando resultados confiables a partir de datos históricos procesados y organizados. Se cumplió con el objetivo de entrenar un modelo capaz de responder a diferentes escenarios, integrando programación, manejo de datos y aprendizaje automático en una solución práctica.

Además, la implementación en Python con librerías especializadas y la estructuración del proyecto en módulos evidencian que el sistema es escalable y transferible a otros contextos. Este trabajo no solo aporta en el ámbito académico, sino que también sienta las bases para futuras aplicaciones en sectores como transporte, energía y agricultura, mostrando que la tecnología puede convertirse en un recurso útil para la sociedad.

BIBLIOGRAFÍA

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Aprendizaje profundo*. MIT Press.

Chollet, F. (2018). *Deep Learning con Python*. Manning Publications.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning en Python*. Journal of Machine Learning Research, 12, 2825–2830.

McKinney, W. (2017). *Python para análisis de datos*. O'Reilly Media.

Hunter, J. D. (2007). *Matplotlib: Una biblioteca 2D de gráficos en Python*. Computing in Science & Engineering, 9(3), 90–95.

National Renewable Energy Laboratory (NREL). (2020). *Archivos climáticos TMYx para simulación energética*. Recuperado de <https://energyplus.net/weather>

Organización Meteorológica Mundial (OMM). (2019). *Guía de datos climáticos y predicción meteorológica*. Ginebra: OMM.